

# Processamento Digital de Sinais: Filtragem FIR e Detecção de Transitório em Corrente Industrial

Nome do Autor 1, Nome do Autor 2, Nome do Autor 3 e Nome do Autor 4

## I. FUNDAMENTAÇÃO TEÓRICA

Explicar o algoritmo de processamento digital de sinais utilizado.

Citar as referências bibliográficas utilizadas para a compreensão da fundamentação teórica do algoritmo. Por exemplo, para citar uma referência que trata de aplicações da *Fast Fourier Transform* (FFT), você deve cadastrar os dados bibliográficos no arquivo ref.bib e chamar a citação da seguinte forma [?].

Detalhar e explicar seu equacionamento. Criar um pseudo-código (use o pacote algorithm) ou um fluxograma (usar uma figura) do algoritmo. A Figura ?? traz um exemplo de fluxograma.

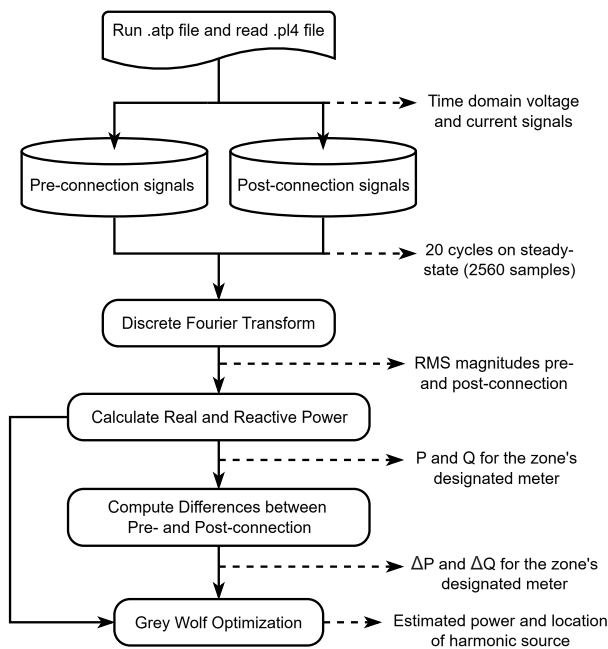


Figura 1. Exemplo de Fluxograma.

Obs.: Será verificado o uso de IA Generativa. Somente será aceito para fins de correção ortográfica/gramatical. Escrita feita por IA Generativa não é permitida.

Obs.: Esse documento deve ter entre 5 e 8 páginas, sem contar o Apêndice.

## II. ANÁLISE DOS SINAIS

Apresente o(s) sinal(is) que seu grupo está utilizando. O sinal de corrente foi amostrado a  $f_s = 7680$  Hz (128 amostras/ciclo  $\times$  60 Hz), totalizando  $N = 1537$  amostras em

aproximadamente 0,20 s (12 ciclos). A Figura ?? apresenta a comparação entre o sinal com ruído, o sinal sem ruído de referência e o sinal após a filtragem FIR ótima.

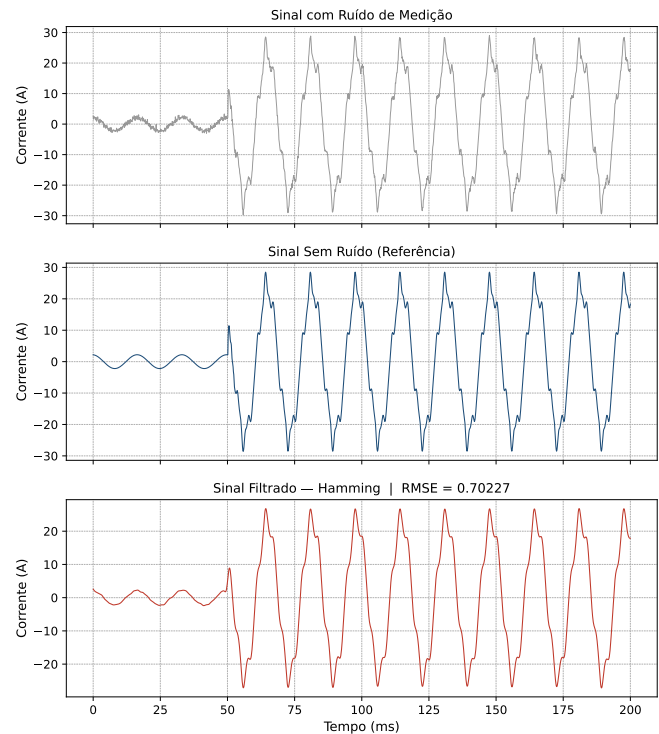


Figura 2. Comparação entre o sinal com ruído de medição, o sinal sem ruído (referência) e o sinal filtrado com o melhor filtro FIR (Janela: Hamming, 32 taps, ordem 31, RMSE = 0,7023 A).

Explique as parametrizações testadas para o algoritmo de processamento digital de sinais utilizado. Foram testadas 3 funções de janelamento (Hamming, Blackman e Kaiser com  $\beta = 8,6$ ) combinadas com 3 ordens de filtro (31, 63 e 127 taps correspondendo a 32, 64 e 128 coeficientes), totalizando 9 configurações. A filtragem foi aplicada com fase zero via `filtfilt` para eliminação do atraso de grupo na avaliação. A Tabela ?? consolida os resultados de RMSE para cada combinação.

Demonstre, de preferência graficamente, as saídas do algoritmo de processamento digital de sinais utilizado. A Figura ?? apresenta o instante de inserção da carga não-linear detectado programaticamente pelo método de RMS deslizante com detector de gradiente máximo.

Apresente e discuta os resultados finais de seu projeto. A partir da Tabela ?? e da Figura ??, o filtro FIR com

Tabela I

RMSE (A) DO SINAL FILTRADO EM RELAÇÃO AO SINAL SEM RUÍDO, POR JANELA E NÚMERO DE TAPS DO FILTRO FIR. MELHOR RESULTADO DESTACADO EM NEGRITO.

| Janela                   | Número de Taps (Ordem) |              |                |
|--------------------------|------------------------|--------------|----------------|
|                          | 32 (ord. 31)           | 64 (ord. 63) | 128 (ord. 127) |
| Hamming                  | <b>0,7023</b>          | 0,7231       | 0,7603         |
| Blackman                 | 0,7658                 | 0,7034       | 0,7457         |
| Kaiser ( $\beta = 8,6$ ) | 0,7663                 | 0,7035       | 0,7458         |

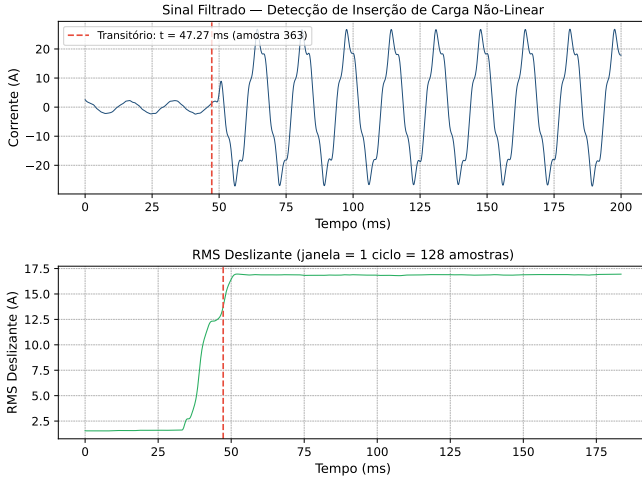


Figura 3. Detecção programática da inserção da carga não-linear. A linha tracejada indica o instante detectado em  $t = 47,27$  ms (amostra 363, ciclo  $\approx 2,84$ ). O painel inferior mostra o RMS deslizante de 1 ciclo (128 amostras) cujo gradiente máximo identifica o transitório.

janela Hamming e 32 taps (ordem 31) apresentou o melhor compromisso entre qualidade de filtragem (RMSE = 0,7023 A) e custo computacional (menor número de multiplicações por amostra).

### III. CONCLUSÕES

Apresente as conclusões obtidas pelo grupo em relação à aplicação do algoritmo.

#### APÊNDICE

##### APÊNDICE: CÓDIGO PYTHON IMPLEMENTADO

Implementação desenvolvida com auxílio de Claude (Anthropic, modelo Sonnet 4.6). Prompt principal: “Execute the full DSP pipeline: FIR filtering with window comparison, transient detection, and LaTeX integration for the PDS final project (SEL 0615).”

```
1 import sys
2 import logging
3 import numpy as np
4 import pandas as pd
5 from scipy.signal import firwin, filtfilt, freqz
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 from pathlib import Path
10
11 logging.basicConfig(
```

```
    level=logging.INFO,
    format="%asctime)s [%s] %(message)s"
    ,
    stream=sys.stdout,
)
16 log = logging.getLogger(__name__)

FS = 7680.0          # Hz      128 samples/cycle    60
                    # Hz
FC = 600.0          # Low-pass cutoff (Hz): preserves
                    # up to 10th harmonic, rejects broadband noise
OUTPUT_DIR = Path("PDS")

21 #
# -----
# Data loading
# -----

26 def load_signal(path: str) -> np.ndarray:
    return pd.read_csv(path, header=None)[0].values

#
# -----
# FIR filter design and application
# -----

31 def design_fir(numtaps: int, window, cutoff: float =
    FC, fs: float = FS) -> np.ndarray:
    """Window-based linear-phase FIR low-pass filter
    (Type I for odd numtaps).

    Group delay of the causal implementation: (
        numtaps - 1) / 2 samples.
    At fs=7680 Hz and numtaps=128, this is 63/7680
        8.2 ms.
    filtfilt cancels this delay for offline
        evaluation; lfilter with explicit
        index shift must be used for causal/real-time
        deployment.
    """
    return firwin(numtaps, cutoff, window=window, fs
        =fs)

41 def apply_filter(signal: np.ndarray, b: np.ndarray)
    -> np.ndarray:
    """Zero-phase FIR filtering via forward-backward
        pass (filtfilt).
    Doubles effective roll-off order; suitable for
        offline RMSE evaluation.
    """
    return filtfilt(b, 1.0, signal)

46 def rmse(filtered: np.ndarray, reference: np.ndarray)
    -> float:
    return float(np.sqrt(np.mean((filtered -
        reference) ** 2)))

#
# -----
# Transient (non-linear load insertion) detection
# -----
```

```

def detect_transient(signal: np.ndarray, fs: float,
                    window_samples: int = 128):
    """Rolling RMS energy estimator with maximum-
        gradient detector.

    Computes 1-cycle (128-sample) rolling RMS using
        the cumulative-sum trick
    (O(N) complexity). The sample with the steepest
        positive RMS gradient
    marks the non-linear load insertion onset.

    Returns
    -----
    idx : int        detected sample index in the
                    original signal
    t : float        corresponding time in seconds
    rms : ndarray    rolling RMS array (length N -
                    window_samples)
    """
    sq = signal ** 2
    cs = np.cumsum(sq)
    # Sliding window sum: cs[i] - cs[i -
        window_samples]
    window_sums = np.empty(len(sq) - window_samples
        + 1)
    window_sums[0] = cs[window_samples - 1]
    window_sums[1:] = cs[window_samples:] - cs[:-
        window_samples]
    rms_arr = np.sqrt(window_sums / window_samples)

    grad = np.diff(rms_arr)
    # argmax gives the index inside grad; shift back
        to original signal index
    peak_in_grad = int(np.argmax(grad))
    # peak_in_grad + 1 aligns to the right edge of
        the window where the jump ends;
    # subtract window_samples // 2 to approximate
        the onset midpoint.
    idx = peak_in_grad + window_samples // 2
    return idx, idx / fs, rms_arr

#
-----
# Plotting helpers
#
-----

def _save_pdf(fig: plt.Figure, name: str) -> Path:
    path = OUTPUT_DIR / name
    fig.savefig(path, format="pdf", bbox_inches="
        tight")
    plt.close(fig)
    log.info("Saved %s", path)
    return path

def plot_comparison(
    t: np.ndarray,
    noisy: np.ndarray,
    clean: np.ndarray,
    filtered: np.ndarray,
    best_label: str,
    best_rmse: float,
) -> Path:
    """Three-panel time-domain comparison plot."""
    fig, axes = plt.subplots(3, 1, figsize=(8, 9),
        sharex=True)

    axes[0].plot(t * 1000, noisy, color="#888888",
        linewidth=0.6, alpha=0.85)

```

```

axes[0].set_ylabel("Corrente (A)", fontsize=11)
axes[0].set_title("Sinal com Ruído de Média",
    ", fontsize=11)
axes[0].grid(True, linestyle="--", linewidth
    =0.4)

axes[1].plot(t * 1000, clean, color="#1f4e79",
    linewidth=0.9)
axes[1].set_ylabel("Corrente (A)", fontsize=11)
axes[1].set_title("Sinal Sem Ruído (Referência)",
    ", fontsize=11)
axes[1].grid(True, linestyle="--", linewidth
    =0.4)

axes[2].plot(t * 1000, filtered, color="#c0392b",
    , linewidth=0.9)
axes[2].set_ylabel("Corrente (A)", fontsize=11)
axes[2].set_xlabel("Tempo (ms)", fontsize=11)
axes[2].set_title(
    f"Sinal Filtrado {best_label} | RMSE =
        {best_rmse:.5f}", fontsize=11
    )
axes[2].grid(True, linestyle="--", linewidth
    =0.4)

fig.tight_layout(pad=1.5)
return _save_pdf(fig, "fig_comparison.pdf")

def plot_transient(
    t: np.ndarray,
    filtered: np.ndarray,
    rms_arr: np.ndarray,
    transient_idx: int,
    transient_time: float,
) -> Path:
    """Two-panel plot: filtered signal + rolling RMS
        with transient marker."""
    t_rms = np.arange(len(rms_arr)) / FS # rolling
        RMS has different length

    fig, axes = plt.subplots(2, 1, figsize=(8, 6),
        sharex=False)

    # Top: full-filtered signal with vertical
        transient marker
    axes[0].plot(t * 1000, filtered, color="#1f4e79",
        , linewidth=0.8)
    axes[0].axvline(----
        x=transient_time * 1000,
        color="#e74c3c",
        linestyle="--",
        linewidth=1.4,
        label=f"Transição: t = {transient_time *
            1000:.2f} ms (amostra {transient_idx})",
    )
    axes[0].set_ylabel("Corrente (A)", fontsize=11)
    axes[0].set_title("Sinal Filtrado Detecção
        de Inserção de Carga Não-Linear",
        , fontsize=11)
    axes[0].legend(fontsize=9, loc="upper left")
    axes[0].grid(True, linestyle="--", linewidth
        =0.4)
    axes[0].set_xlabel("Tempo (ms)", fontsize=11)

    # Bottom: rolling RMS with gradient peak marker
    axes[1].plot(t_rms * 1000, rms_arr, color="#27
        ae60", linewidth=0.9)
    axes[1].axvline(
        x=transient_time * 1000,
        color="#e74c3c",
        linestyle="--",
        linewidth=1.4,
    )

```

```

171 axes[1].set_ylabel("RMS Deslizante (A)",
    fontsize=11)
172 axes[1].set_xlabel("Tempo (ms)", fontsize=11)
173 axes[1].set_title("RMS Deslizante (janela = 1
    ciclo = 128 amostras)", fontsize=11)
174 axes[1].grid(True, linestyle="--", linewidth
    =0.4)

175 fig.tight_layout(pad=1.5)
176 return _save_pdf(fig, "fig_transient.pdf")

#
-----

# Main pipeline
#
-----

def main() -> None:
    log.info("Loading signals      fs=%.0f Hz, fc=%.0
        f Hz", FS, FC)
    noisy = load_signal("sinais/sinal_2_ruido.csv")
186 clean = load_signal("sinais/sinal_2_semruído.csv
    ")
    assert len(noisy) == len(clean), "Signal length
        mismatch"
    N = len(noisy)
    t = np.arange(N) / FS
    log.info("N=%d samples, duration=%.4f s (~%.1f
        cycles)", N, N / FS, N / (FS / 60))

191 # --- FFT spectrum summary (informational) ---
    freqs = np.fft.rfftfreq(N, d=1.0 / FS)
    noise_spectrum = np.abs(np.fft.rfft(noisy -
        clean))
    peak_noise_freq = freqs[np.argmax(noise_spectrum
        )]
196 log.info("Peak noise energy at %.1f Hz (choose
        fc=%.0f Hz to reject)", peak_noise_freq, FC)

    # --- FIR filter test matrix ---
    windows = [
        ("hamming",          "Hamming"),
201        ("blackman",        "Blackman"),
        (("kaiser", 8.6),    "Kaiser ( =8.6)"),
    ]
    numtaps_list = [32, 64, 128] # orders 31, 63,
        127

206 results = []
    print(f"\n{'Window':<20} {'Taps':>6} {'RMSE
        ':>12}")
    print("-" * 42)
    for taps in numtaps_list:
        for win_param, win_name in windows:
211            b = design_fir(taps, win_param)
            filtered = apply_filter(noisy, b)
            err = rmse(filtered, clean)
            group_delay_ms = (taps - 1) / 2 / FS *
                1000
            results.append({
                "window": win_name,
                "win_param": win_param,
                "numtaps": taps,
                "order": taps - 1,
                "rmse": err,
                "group_delay_ms": group_delay_ms,
                "filtered": filtered,
            })
221            print(f"{win_name:<20} {taps:>6} {err
                :>12.6f}")

226 # --- Select best configuration ---
    best = min(results, key=lambda r: r["rmse"])
    log.info(
        "Best filter: %s, %d taps (order %d), RMSE
            =%.6f, group delay=%.2f ms",
        best["window"], best["numtaps"], best["order
            "], best["rmse"], best["group_delay_ms"
            ]
    )

    # --- Transient detection ---
    tr_idx, tr_time, rms_arr = detect_transient(best
        ["filtered"], FS)
    log.info(
        "Transient detected: sample %d, t=%.4f s
            (%.2f ms), cycle ~%.1f",
        tr_idx, tr_time, tr_time * 1000, tr_time *
            60,
    )

    # Cross-validate on clean signal
    tr_idx_clean, tr_time_clean, _ =
        detect_transient(clean, FS)
    log.info(
        "Cross-validation on clean signal: sample %d
            , t=%.4f s",
        tr_idx_clean, tr_time_clean,
    )

    # --- Plots ---
    OUTPUT_DIR.mkdir(exist_ok=True)
    plot_comparison(t, noisy, clean, best["filtered"
        ], best["window"], best["rmse"])
    plot_transient(t, best["filtered"], rms_arr,
        tr_idx, tr_time)

    # --- Structured summary for LaTeX injection ---
    print("\n=== LATEX TABLE DATA ===")
    print(f"{'Window':<22} {'32 taps':>10} {'64 taps
        ':>10} {'128 taps':>10}")
    print("-" * 54)
    for _, win_name in windows:
        row_vals = [r["rmse"] for r in results if r[
            "window"] == win_name]
        vals_str = " ".join(f"{v:.5f}" for v in
            row_vals)
        print(f"{win_name:<22} {vals_str}")

    print(f"\n=== TRANSIENT DETECTION ===")
    print(f"  Filtered signal: sample={tr_idx}, t={
        tr_time:.4f} s ({tr_time*1000:.2f} ms),
        cycle {tr_time*60:.2f}")
    print(f"  Clean signal: sample={tr_idx_clean
        }, t={tr_time_clean:.4f} s ({tr_time_clean
        *1000:.2f} ms), cycle {tr_time_clean*60:.2
        f}")
    print(f"\n=== BEST FILTER ===")
    print(f"  Window: {best['window']}")
    print(f"  Taps: {best['numtaps']} (order {best['
        order']})")
    print(f"  RMSE: {best['rmse']:.6f}")
    print(f"  Group delay (lfilter): {best['
        group_delay_ms']:.2f} ms")

    if __name__ == "__main__":
271         main()

```

Listing 1. Implementação completa do pipeline DSP em Python (main.py).